

揭秘高德地图动态化跨端容器--AJX

AJX是什么

AJX是 [A]map, [J]ava Script, [X]ML 三个单词的首字母缩写，是高德客户端基于当时对移动互联网重云轻端、云端一体这一技术发展趋势的判断而自研的基于 JavaScript 引擎、用于实现移动端跨平台应用开发的系统化解决方案。AJX具有接近于原生应用的高性能，能力上支持可扩展，且兼具热更新的特点，以一套代码，一端开发、一遍测试，动态发版，接近原生 Native 的性能体验为设计宗旨。早期在团队内部沟通时，也常常称呼为动态UI（或称为AJX容器），AJX的一个重要技术特色是提供较为灵活的动态化UI的交付能力（相对于纯原生 Android/Java 或 iOS/Objective C 开发）。

AJX产生的背景

提供架构解决方案时，识别问题往往比解决问题更重要，对于问题的识别将深刻影响后期的进一步决策。

2015年伴随着高德数据生产、数据服务改造完成后，客户端作为一个流量入口其迭代效率成了高德业务发展的瓶颈，而在同一个阶段高德地图移动客户端Amap在iOS/Android上缺少统一架构，模块间的耦合比较严重，牵一发而动全身，产/研/测的成本及压力大，客户端的发版周期特别长；为顺应当时客户端架构的潮流，且Amap也要顺势逐渐由工具类产品升级为平台级的产品的战略，为云端一体化，千人千面、ABTest做好提前布局也成为了当时的考量。



2015年，市面上已经有了Facebook推出的React Native类似的跨端方案，同时集团内的Weex也在研发当中，H5本质上也是为了跨端而设计的，但我们当时的判断是：Weex、React Native的存在证明了这样的架构是可行的，另一方面高德业务的高速发展，会要求技术架构随时能应对业务发展的变化，React Native是一个通用化的解决方案，但它不一定适合Amap的业务，尤其是对于高德地图这种富交互的产品，作为一个亿级用户量的地图应用，我们不可能把自己的命交到一个正在发展中的开源项目手上，而H5的交互体验则和原生相差太远，对于Amap中很多场景尤其是出行场景在当时来看也不太可能选择H5作为主流的业务架构。

在技术选型上，以前端语言JavaScript为DSL，引入yoga flex来进行排版布局，用C++实现Web容器的DOM API，并通过JS binding的方式透出给业务层，C++解析DSL中的前端标签，进而进行布局计算，最终以事件的形式通知到原生Android/iOS端，通过原生基础组件的来表达DSL中的组件标签，尽可能的保持Amap UI具备原生的体验，对于Android/iOS端平台如键盘、蓝牙等平台相关的能力则可通过桥接的方式由原生来补充实现，由此便可以达成用JavaScript一端实现业务逻辑，在Android/iOS双端运行的目的。

回到问题，要实现一个满足于Amap的跨端容器，需要实现以下几项基础能力：JavaScript运行环境、原生组件渲染能力、对地图的能力、动态化能力、调试能力。

AJX容器发展的初期

因为相信所以看见，选择正确的方向，便会吸引有共同理念的伙伴加入，最终时间会证明一切。

在2016年底，从各业务团队抽调骨干人力，启动了AJX 3.0 专项建设，在之前我们有过AJX1.0, 2.0，但真正意义的容器化的AJX是从3.0开始，我们从0到1开始建立AJX3.0的基础能力：

JS运行环境：是指实现业务需要的JS 执行环境，通过JS binding的方式提供布局计算、动画、存储、页面通信，平台能力等API环境。

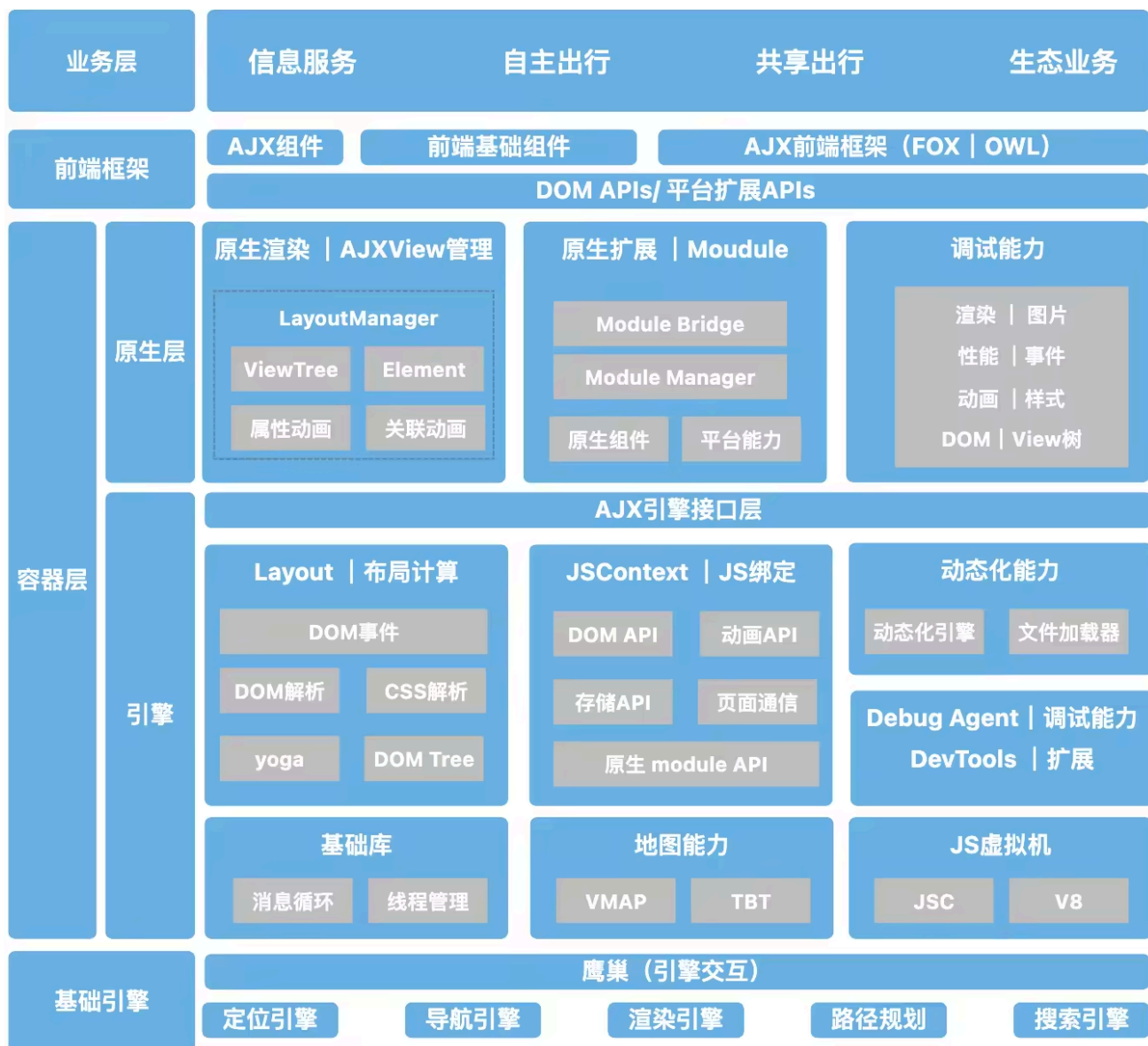
原生组件渲染能力：是指使用原生组件表达前端组件标签，并提供动画、事件响应能力；

地图能力：是指对接地图渲染、导航等地图能力为业务层实现JS API，期间我们同步启动了C++ 部分专项工作：鹰巢，它将原Amap中的地图基础引擎进行了抽象和封装，使得上层业务AJX能更好的使用地图能力；

动态发版能力：为动态化我们实现了AJX文件格式及配套打包工具，在端上实现了AJX文件加载器及同服务端的支撑侧同学一起实现的AJX 发版能力；

调试能力：是指IDE和手机之间的调试通道，协议，以及以此为基础建立的调试工具集（JS/DOM/日志等调试能力）；

基于以上便有了Amap AJX容器的最初的技术架构，关于AJX架构更详细的介绍请参见 [一文读懂AJX容器架构](#)。



经过几个月的艰苦奋斗，随着POI（信息业务）成为AJX3.0落地的第一个复杂的业务，高德移动端的开发模式迎来新的生机，到2017年AJX 3.0 发布，经统计信息POI业务的 AJX 代码规模为 3W 行，同期，高德打车（今天的共享业务）、公共出行业务 和 基础地图 也开始采用 AJX 开发，AJX3.0未来可期。

在AJX早期支持业务中，由于前端、原生开发、C++开发几端存在天然的技术栈壁垒，排查问题往往需要跨几端打日志才得以定位，大多数业务同学从原生开发转到AJX，顶着需求进度的压力，在AJX初期又没有详尽的开发文档以及调试工具、业务同学往往需要同AJX支撑同学当面沟通，以口口相传等低效的方式学习一些新接口的使用。

那段时间虽然煎熬但也是跨团队配合最默契的时候，与此同时，大家更多的是收获了技术上的成长，现在回忆起来，大家能一路坚定信心的迈过这一关键时期，真的是应了阿里那句老话：因为相信，所以看见！感谢一路背靠背战斗的小伙伴们！

AJX容器发展的发展期

工欲善其事，必先利其器。AJX的引入是为了提升Amap的研发效率，工具链的够用、稳定、易用是AJX得以真正助力业务发展的前提。

2018年到2019年，随着自主出行，共享出行，信息业务的全面接入AJX，越来越多的原生开发人员加入AJX的开发者阵营，随着业务逐步自然演进，业务部分代码属于仍然属于原生代码，导致这些业务骑兵还是不能被解放天性真正快起来，这些问题开始成为影响产品功能迭代效率的新瓶颈。经过架构团队、业务团队讨论，最后得出的共同结论是：迅速启动业务完整上漂AJX，核心能力下沉C++，业务开启以五独化（独立编译、独立构建、独立发布、独立测试、独立运营）为目标的研发模式。

对于业务线的开发者来说，在原生开发接触到的开发文档，工具经验不再继续适用于AJX，对大多数业务线同学来说AJX是一套全新的技术栈，作为AJX支撑团队，为了让业务在五独模式下像骑兵一样的释放自己的高灵活、快速度的天性，AJX必须为他们提供良好的技术支撑，如工具链：代码编辑器，编译器，打包工具，调试工具，AJX开发者文档，以及对于新能力诉求、常见问题接发的快速响应，毕竟做AJX不是为了完成一个KPI，而是真正为开发者提高生产力，助力业务快速发展，让骑兵冲起来！

在这个时期，为业务提供完善的工具链支持成为了我们的工作重点，AJX的工具链建设主要分为编译工具、代码编写、调试工具、社区维护四大部分：



工具链的相关内容本身可以自成一个分享体系，感兴趣的同学可进一步关注我们关于[《AJX容器工具链建设》](#)系列文章。

AJX容器的架构稳固期

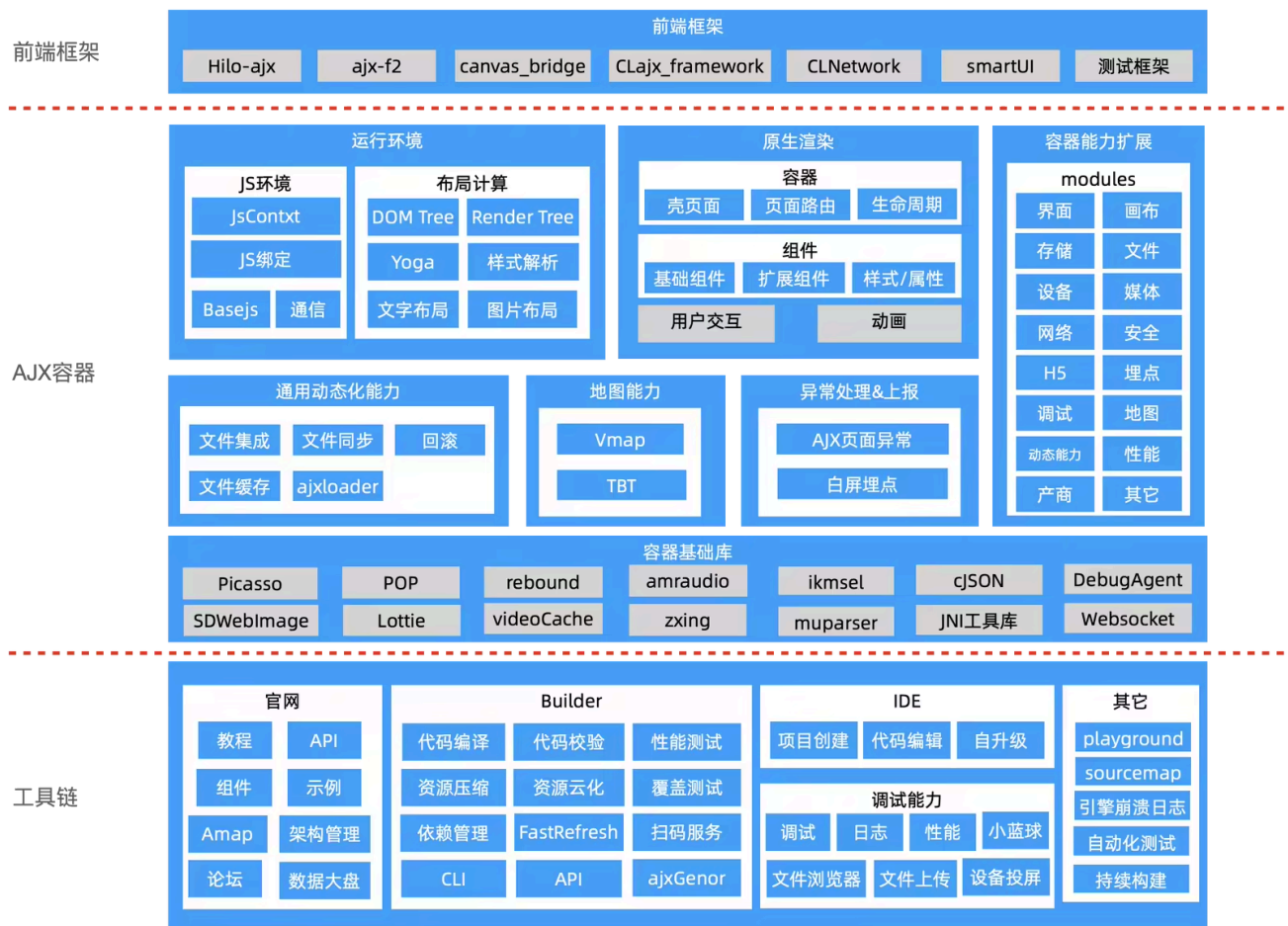
没有完美的架构，只有刚好的架构，没有满足一切的架构，只有满足目标的架构，任何优秀的架构都是在业务的实践和迭代中逐渐完善的。在业务的实践中不惧任何困难，遇到问题见招拆招，进一步优化并解决它，是每一个架构师应该持之以恒的追求。

AJX的另外一个大的优点是动态化能力，但在业务不断的迭代中，我们发现虽然AJX已具备动态化能力，但因为业务模块之间依赖耦合严重，任何业务的发布都需要层层关联和捆绑，导致当时我们为做不同业务模块的热更生效顺序和时机复杂到需要依赖无向图找环算法来解题。

2021年，借由架构升级和包大小专项，联合业务团队，我们启动了业务模块的依赖治理，通过动态引擎升级（[《动态引擎的升级见高德地图AJX容器动态引擎策略进化之路-举起自己》](#)）使业务在使用动态化能力上成本进一步降低，并且升级后业务可以根据自己的特点

来进行除AJX以外的地图资源，Android SO动态库，Android插件的全云化。依赖的治理除了让业务的架构更清晰，也给Android 的包大小带来了十分亮眼的收益：Android 安装包由150M 降低为 90M。一套代码多端运行，动态化的发布链路成熟，AJX正式进入稳定期。

从容器能力以及工具链的支撑上看，当前AJX容器的能力支撑也逐渐趋于完善：



2021下半年开始，我们开始重新捡起AJX在随业务快速发展期遗留的工作，如：API的标准化[《高德地图AJX容器API标准化演进之路》](#)，容器线上数据预警，性能优化（预加载能力，UI线程布局渲染能力，UI 执行JS代码的能力：实现函数式动画计算，List优化），在这一阶段也逐渐开始将AJX容器落地更多新业务场景，如：《基于AJX跨端技术实现高德地图主题换肤》，如：大屏 /横屏/分屏适配能力。

AJX容器未来的展望

2022年，随着高德业务的快速发展，据统计AJX总的代码量从2019年的几万行，已迅速发展到了2022年的数百万行，这说明AJX已经支撑起了业务的高速发展。

接下来，我们会继续解决伴随业务高速发展过程中背负的历史包袱以及面临的新的挑战，如：业务对于动态化能力在使用上的差异化诉求，对于AJX极致性能体验的追求，以及满足AMAP新的技术栈形式的下工具链建设，架构自身的防腐化（Scheme的治理），AJX容器经验的沉淀和拉通（AJX、H5容器API标准化）等工作。

敢于迎接未来，开放心态、取长补短，这一点，与所有架构师共勉！

关于H5和小程序，于Amap而言，它们和AJX之间并非竞争关系而是对移动端跨端技术的相互补充，它们各自满足了不同的Amap业务形态，任何技术最终都是为了达成业务更快、更好的发展，AJX在研发效率上高于原生开发但性能体验上又好于H5，这就是它于Amap业务发展过程中的核心价值，而小程序、H5对于API的标准化及工具链的建设是AJX一直要持续学习的部分。

今天AJX支撑团队的核心能力绝不再是仅仅因为有了有一个能支撑Amap能继续跑3-5年的跨端容器，而是我们7年以来在支撑业务团队时所积累的移动端跨端容器架构如何适应业务迭代的经验、客户端API标准化演进过程中积累的经验、以及培养跨端容器支撑人才的能力，这些才是我们整个AJX架构、业务团队最宝贵并可长期复用的经验！

写在最后的话

本文沿着AJX的不同的发展阶段，简要介绍了在AJX能力、工具链、业务建设中的一些关键时间节点和事件，总结一下：AJX容器的产生是为了解决高德地图Amap的迭代效率问题，AJX基础能力建设并应用于POI的上线，证明AJX容器对于高德业务的能力支撑是可行的；AJX业务五独和AJX工具链深入建设代表着AJX容器进入高速发展阶段；业务模块的依赖解耦和动态化能力的升级代表着AJX容器动态化能力进一步成熟，AJX也进入了架构的稳固期。

6年来，AJX是幸运的，2016年以来高德客户端的业务、服务支撑、产品团队给予了AJX容器长期的支持和包容，尤其是业务团队，他们在落地具体的业务时给AJX容器的发展提供了很多建设性的意见。AJX支撑团队把支撑好业务当最高优先级，只有成就了支撑的业务才能更好的成就自己，同时我们也坚守了【哪些应该架构上解决、哪些应该业务逻辑上解决】的架构规则。正是所有小伙伴们这种像对待自己孩子一样的心态，AJX才能一路走来并茁壮成长起来。**我们成就了彼此，更应该感谢彼此！**

本文对技术细节没有详细展开，喜欢干货的同学可能觉得不过瘾，为此，我们近期会推出关于AJX的系列专题文章，敬请感兴趣的同学关注，大家也可以随时和我一起探讨，欢迎大家一起关注AJX技术和团队。

相关文章

AJX容器

[《揭秘高德地图动态化跨端容器--AJX》](#)

[《一文读懂高德地图AJX容器架构原理》](#)

容器能力

[《从长列表看高德地图AJX容器能力演进》](#)

动态化方案

[《高德地图AJX容器动态引擎策略进化之路-举起自己》](#)

[《高德地图通用动态引擎面临的挑战与思考》](#)

[《高德架构升级：客户端动态框架篇》](#)

[《图算法在高德地图bundle生效逻辑中的探索应用》](#)

工具链建设

[《高德地图AJX容器工具链建设》](#)

[《在DevTools中显示JSC引擎的内存分布》](#)

[《高德 APP 全链路源码依赖分析工程》](#)

[《从依赖分析到循环引用处理——编译原理、算法与数据结构》](#)

[《高德AJX远程调试揭秘》](#)

[《高德客户端日志分析工具-bug效能工具》](#)

[《Amock平台-围绕高德业务崭露头角》](#)

体验优化

[《高德地图体验优化AJX容器篇》](#)

[《高德地图体验优化AJX业务篇》](#)

[《AJX容器切换JavaScriptCore引擎》](#)

质量建设

[《AMap性能问题系统化定位方案》](#)

[《高德AJX支撑基础功能自动化测试》](#)

[《高德AJX质量保障方案》](#)

标准化

[《高德地图AJX容器API标准化演进之路》](#)

[《高德H5容器建设总结》](#)

动效

[《高德地图AJX动效前世今生及思索下一代交互形态》](#)

前端框架

[《状态驱动界面——Fox 框架开发业务的推荐编程范式》](#)

效能

[《高德前端搭建平台系列（1）-物料侧能力建设》](#)

[《深入高德 Acorn Sketch》](#)

[《高德FC函数端云一体》](#)

[《Acorn-设计&研发一体化》](#)

[《Acorn物料平台》](#)

业务实践

[《基于AJX跨端技术实现高德地图主题换肤》](#)

[《像素知多少&AJX上的适配问题》](#)

[《高德地图 AJX bundle 化实践总结》](#)

[《高德地图长列表优化实践》](#)

[《高德地图图面气泡上云的思考与实践》](#)

[《高德地图场景化业务上云实践》](#)

[《公交一体化的前世今生》](#)

业务思考

[《AJX学习与感想》](#)

[《初识AJXBuilder》](#)

[《FOX学习心得》](#)

【附】AJX3.0记事

•

2016年底，启动 AJX 3.0 专项建设，POI（信息业务）同步采用 AJX 开发

•

2017年，AJX 3.0 发布，经统计 信息POI业务的 AJX 代码规模为 3W 行。同期，高德打车（今天的共享业务）、公共出行业务 和 基础地图 也开始采用 AJX 开发

•

2018年，自驾（驾车业务）由双端 Native 代码改为 AJX 和 C++ 技术栈，C++ 部分代号为 鹰巢。同期，基于 vscode 的 AJX IDE 项目启动，实现编写、构建、调试等开发活动一体化

•

2019年，共享业务，信息业务，部分 出行业务 的 Native 代码全面改造为 AJAX，代号 五独 专项（开发、集成、测试、发布、运维），到年底，AJAX 代码规模达 90W

-

2020年，App启动优化 专项完成。经过 五独 和 启动优化，基于 AJAX 技术的业务的 性能体验 和 迭代效率 有较为明显的提升

-

2021年，架构升级和包大小 专项，Android 安装包由 150M 降低为 90M，动态化的发布链路成熟。到年底，AJAX 业务代码规模突破 260W。AJAX 内置 Binary Size 为 34M，云上 Binary Size 为 95M，大部分 AJAX 业务均已上云，用户即用即下